

Binary label symmetry gives a small counterexample

Disproof of Conjecture 00000001469

ziangni-sys

10 October 2026

Result. Under the stated literal requirement of a unique two-label solution, a two-vertex graph is already a counterexample. Global label complementation gives a second distinct solution of exactly the same value for every binary Unique Game.

Scope of the source statement

The source asserts a combinatorial form in which every graph admits a near-optimal game with a unique two-label solution, and states that, if this fails, the minimal counterexample has more than 2^{40} vertices. The Chinese version has the same unique two-label solution requirement and the same numerical lower bound. Neither version pins a vertex label or takes solutions modulo simultaneous label interchange.

We interpret “unique solution” as a unique complete labeling, and “two-vertex game” as pairwise edge constraints, while also giving a graph with exactly two vertices. This concerns the literal combinatorial assertion in this problem. It is not a refutation of the standard Unique Games Conjecture about computational hardness: “unique” in a usual Unique Game refers to the edge permutation, not uniqueness of a global optimum.

The actual binary game and its symmetry

Let $G = (V, E)$ be any finite nonempty graph. A binary Unique Game assigns to each oriented edge uv a permutation π_{uv} of the label set $\{0, 1\}$, with $\pi_{vu} = \pi_{uv}^{-1}$. A labeling $a : V \rightarrow \{0, 1\}$ satisfies uv exactly when

$$\pi_{uv}(a(u)) = a(v).$$

Let $J(b) = 1 - b$. Every permutation of a two-element set is either the identity or J , and therefore commutes with J . Define the complemented labeling $\bar{a} = J \circ a$. Then, edge by edge,

$$\pi_{uv}(\bar{a}(u)) = \bar{a}(v) \iff J(\pi_{uv}(a(u))) = J(a(v)) \iff \pi_{uv}(a(u)) = a(v).$$

Thus a and $\bar{a}$ satisfy exactly the same edges. Their number or fraction of satisfied edges is identical. This also holds for arbitrary fixed edge weights, although the unweighted case already suffices.

Since V is nonempty and J has no fixed label, $\bar{a} \neq a$. Consequently every optimal labeling has a distinct second optimal labeling. More generally, for any score threshold q , if a is a qualifying near-optimal labeling, then so is $\bar{a}$, with exactly the same score. If there is no qualifying labeling, there is also no unique qualifying labeling. Hence no binary edge-permutation game on a nonempty graph can have exactly one optimal or score-based near-optimal labeling.

The small graph and the numerical contradiction

Take the graph with two vertices and its one possible edge. It admits genuine binary Unique Games, for example the identity permutation constraint, so the class is nonempty. The preceding argument applies to every possible choice of its edge permutation, rather than only to one unfavorable instance. Therefore this graph cannot admit the unique two-label solution required in the source.

Its vertex count is 2, and

$$2 \leq 2^{40} = 1099511627776.$$

Thus a counterexample already exists with at most two vertices; a minimal counterexample, if phrased that way, cannot have more than 2^{40} vertices. This disproves the claimed lower bound. Label pinning or quotienting by global complement could remove this elementary obstruction, but those are additional conventions absent from the source.

Formal verification

Lean defines actual binary Unique Games on simple graphs with label-permutation edge constraints and the inverse-orientation condition. It proves that all permutations of the two labels commute with complementation, verifies edge-by-edge invariance, and defines the actual finite score by summing satisfied oriented edges. Counting both orientations simply doubles the usual undirected score. It proves the complement is distinct on every nonempty vertex set and rules out a unique optimal or threshold-qualified labeling for every game. It constructs the complete two-vertex graph with a genuine constraint instance, specializes the universal obstruction to all games on two vertices, and checks the numerical bound.

Run `lake build` in `lean/`. The project pins Lean 4.19.0 and Mathlib commit

`c44e0c8ee63ca166450922a373c7409c5d26b00b`

Final theorem axiom audits are printed. No admitted results, custom axioms, native unchecked decisions, or unsafe code are used.